

Title:

Implement a Neural Network with Mini-Batch Gradient Descent and Learning-Rate Decay (From Scratch)

Objective:

You will implement a **two-layer neural network** (1 hidden layer + 1 output layer) using **pure Python** (no NumPy, no ML libraries), and train it on the **XOR classification problem** using:

- Mini-Batch Gradient Descent, and
 - Time-based Learning Rate Decay.
-

Requirements

1. Network Architecture

- Inputs: 2 features (for XOR)
- Hidden layer: at least 2 neurons
- Output layer: 2 neurons (one-hot encoded classes: [1,0] or [0,1])
- Activation: **sigmoid** for all neurons

2. Dataset (XOR)

x1	x2	label
0	0	0
0	1	1
1	0	1
1	1	0

3. Training Method – Mini-Batch + LR Decay

- Use a mini-batch size (e.g., 2).
- For each epoch:
 1. Compute a **decayed learning rate** using a time-based schedule:

$$\eta_{epoch} = \frac{\eta_0}{1 + decay \cdot epoch}$$

where

η_0 = initial learning rate,

decay = a small constant (e.g., 0.001).

2. Shuffle the dataset at the start of the epoch.
3. Split data into mini-batches.
4. For each batch:
 - Forward pass each sample → outputs
 - Backpropagate → deltas

- Accumulate gradients across samples in the batch
- After the batch, **update weights once** using **average gradients** and current decayed learning rate.

4. Outputs

- Print:
 - epoch number
 - current decayed learning rate
 - total training error (sum of squared errors)
- After training:
 - Print XOR inputs, true labels, predicted labels.

5. Goal

By completing this assignment, you will:

- Understand **mini-batch gradient descent** implementation details.
- See how a **decaying learning rate** helps stabilize training:
 - Large steps at the beginning,
 - Smaller, more careful steps later.